

C Programming Projects

Combined Documentation

Reverse | Unix Utilities | Wish Shell

A practical walkthrough of the design, main logic, errors, compilation, and testing.

Prepared for the three C/Unix programming projects

Contents

1. Overview	3
2. Project 1 - Reverse	4
3. Project 2 - Unix Utilities	7
4. Project 3 - Wish Shell	12
5. Final Notes	18

How to use this document

This report explains what each project does and how the implementation is put together. It is written as practical documentation rather than a formal research paper. Code examples are kept short so the focus stays on the main ideas.

Note: The command examples assume a Linux or other Unix-like terminal and GCC.

1. Overview

These three projects build up from basic file handling to process management. The first project reverses lines of text. The second creates four small Unix-style tools. The third is a working command shell that launches programs and supports built-in commands, redirection, and parallel execution.

Project	Main topic	Key C features	Output
Reverse	Reading and storing text	getline, malloc, linked lists, files	Lines printed in reverse order
Unix Utilities	Small command-line tools	fopen, fgets/getline, fwrite/fread	cat, grep, compression, decompression
Wish Shell	Processes and shell behavior	fork, execv, waitpid, chdir, dup2	Interactive or batch command execution

Shared goals

- Use command-line arguments correctly.
- Check return values from file, memory, and process-related functions.
- Print the required error message exactly when the assignment specifies one.
- Release allocated memory and close opened files.
- Compile cleanly with warnings enabled.

```
gcc -Wall -Werror source_file.c -o program_name
```

2. Project 1 - Reverse

The Reverse program reads text and prints its lines from last to first. It can read from the keyboard or from a file, and it can write either to the screen or to an output file.

2.1 Ways to run it

```
./reverse
./reverse input.txt
./reverse input.txt output.txt
```

- No arguments: read from standard input and write to standard output.
- One argument: read from the named file and print to the screen.
- Two arguments: read from the first file and write the reversed result to the second file.

2.2 Example

Input:

```
hello
this
is
a file
```

Output:

```
a file
is
this
hello
```

2.3 Main design

The program uses a linked list. Every newly read line is inserted at the front of the list. That means the list is already reversed by the time the input ends, so printing from the head gives the correct result.

1. Choose the input and output streams from the command-line arguments.
2. Read one full line at a time with `getline()`.
3. Allocate a linked-list node for the line.
4. Insert the new node at the beginning of the list.
5. After reaching the end of the input, walk through the list and print each stored line.
6. Free every line and node, then close files that were opened.

```
new_line->text = line;
new_line->next = head;
head = new_line;
```

Why this works: Adding each line to the front avoids a separate reversing pass.

2.4 Handling long input

The assignment does not allow a fixed maximum line length. `getline()` solves this by growing its buffer when needed. The number of lines is also not fixed, so the linked list grows one node at a time.

2.5 Error handling

Situation	Required result
Too many arguments	usage: reverse <input> <output>
Input and output are the same	Input and output file must differ
A file cannot be opened	error: cannot open file 'filename'
Memory allocation fails	malloc failed

Errors are sent to stderr and the program exits with status 1. Successful runs return 0.

2.6 Compilation and testing

```
gcc -Wall -Wextra -Werror -std=gnu11 reverse.c -o reverse
printf "hello\nthis\nis\na file\n" > input.txt
./reverse input.txt
./reverse input.txt output.txt
cat output.txt
```

Useful extra tests include an empty file, a file with one line, a file whose final line has no newline, a very long line, a missing file, and identical input/output paths.

2.7 What this project practices

- Dynamic memory allocation
- Linked-list basics
- Standard input and output
- File opening and closing
- Exact error handling

3. Project 2 - Unix Utilities

This project contains four separate C programs: my-cat, my-grep, my-zip, and my-unzip. Each one copies a small piece of behavior from a common Unix command.

3.1 my-cat

my-cat prints the contents of one or more files. Files are processed from left to right in the same order they appear on the command line.

```
./my-cat file1.txt file2.txt
```

7. Loop through every filename in argv.
8. Open the file in read mode.
9. Read chunks with fgets().
10. Print each chunk exactly as it was read.
11. Close the file and continue.

```
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer);
}
```

A fixed buffer is acceptable here because fgets() can read a long line in multiple pieces. The output still remains unchanged.

Error: If a file cannot be opened, print "my-cat: cannot open file" and return 1.

3.2 my-grep

my-grep checks each input line for a case-sensitive search term. Lines containing the term are printed; all other lines are skipped.

```
./my-grep foo notes.txt
printf "foo\nbar\nfood\n" | ./my-grep foo
```

12. Read the search term from argv[1].
13. Use stdin when no filenames are provided.
14. Otherwise open each file in order.
15. Read complete lines with getline().
16. Use strstr() to check whether the term appears in the line.

```
if (strstr(line, term) != NULL) {
    printf("%s", line);
}
```

getline() is useful here because the assignment specifically requires support for lines of any length.

Errors: Missing search term: "my-grep: searchterm [file ...]". File open failure: "my-grep: cannot open file".

3.3 my-zip

my-zip compresses input using run-length encoding (RLE). A run of repeated characters becomes a binary integer count followed by one character.

```
aaaaaaaaabbbb -> 10a4b (logical view)
```

The real file is binary, not the text shown above. Each entry is exactly an int followed by one char.

```
fwrite(&count, sizeof(int), 1, stdout);
fputc(current, stdout);
```

17. Keep the current character and its run count.
18. Increase the count when the next character matches.
19. When the character changes, write the previous count and character.
20. Continue across file boundaries so several files act like one input stream.
21. Write the final run after all files have been read.

```
./my-zip file.txt > file.z
```

Important: Compressed output should be redirected to a file because it contains binary bytes.

3.4 my-unzip

my-unzip reads the binary pairs created by my-zip. For every pair, it prints the stored character the requested number of times.

```
while (fread(&count, sizeof(int), 1, fp) == 1) {
    fread(&ch, sizeof(char), 1, fp);
    for (int j = 0; j < count; j++) {
        fputc(ch, stdout);
    }
}
./my-unzip file.z > restored.txt
```

3.5 How zip and unzip fit together

Stage	Data	Example
Original text	Normal characters	aaabb
Compressed	Binary count + character	3,a,2,b
Uncompressed	Normal characters again	aaabb

3.6 Compilation

```
gcc -o my-cat my-cat.c -Wall -Werror
gcc -o my-grep my-grep.c -Wall -Werror -std=gnu11
gcc -o my-zip my-zip.c -Wall -Werror
gcc -o my-unzip my-unzip.c -Wall -Werror
```

3.7 Practical test set

```
printf "alpha\nbeta\n" > one.txt
printf "beta\ngamma\n" > two.txt

./my-cat one.txt two.txt
./my-grep beta one.txt two.txt
./my-zip one.txt two.txt > data.z
./my-unzip data.z > restored.txt
cat restored.txt
```

- Test no-argument behavior for every program.
- Test a missing input file.
- Test empty files and files with no final newline.
- Test repeated characters that continue from the end of one file into the start of another.
- Compare the restored output with the original combined input.

3.8 What this project practices

- Processing multiple command-line files
- Text versus binary input/output
- Reading from stdin and writing to stdout
- Simple string searching
- Run-length encoding

4. Project 3 - Wish Shell

Wish is a small command shell. It reads a command, decides whether it is built in or external, starts programs, waits for them, and then reads the next command. It supports both interactive use and commands loaded from a batch file.

4.1 Interactive and batch mode

```
./wish
./wish batch.txt
```

- Interactive mode reads from stdin and prints the prompt "wish> ".
- Batch mode reads commands from the provided file and does not print a prompt.
- End-of-file closes the shell normally.
- More than one batch-file argument is an error.

4.2 Main loop

The main loop prints the prompt when needed, reads a complete line with `getline()`, and sends the line to the parser. The default executable search path is `/bin`.

```
while (1) {
    if (interactive) {
        printf("wish> ");
        fflush(stdout);
    }

    if (getline(&line, &capacity, input) == -1) {
        break;
    }

    process_line(line);
}
```

4.3 Parsing a command line

A line can contain several commands separated by `&`. Each command may also contain a single output redirection operator `>`. Normal command arguments are split on spaces and tabs.

22. Split the full line at ampersands to identify parallel commands.
23. Trim extra whitespace from each command.
24. Check for redirection and validate its syntax.
25. Split the command into argv-style arguments.
26. Handle built-in commands directly or launch an external process.
27. After all external commands have started, wait for every child process.

Whitespace: Commands such as "ls>out", "ls > out", and " ls -la " should still parse correctly.

4.4 Built-in commands

Command	Rules	Action
exit	No arguments allowed	Free shell resources and exit with status 0
cd directory	Exactly one directory	Call <code>chdir(directory)</code>
path [dir ...]	Zero or more directories	Replace the entire executable

		search path
--	--	-------------

Built-ins run inside the shell process. Running `cd` in a child process would not change the shell's own current directory, which is why it must be handled directly.

4.5 Finding and launching external commands

For an external command such as `ls`, the shell joins each path directory with the command name and uses `access(..., X_OK)` to check whether the result is executable. The first match is launched.

```
snprintf(full_path, length, "%s/%s", shell_paths[i], command);
if (access(full_path, X_OK) == 0) {
    return full_path;
}
```

The shell then calls `fork()`. The child process calls `execv()`, while the parent stores the child PID so it can wait later.

```
pid_t pid = fork();
if (pid == 0) {
    execv(executable, arguments);
    _exit(1);
}
```

4.6 Output redirection

The `>` operator sends both standard output and standard error to the same file. The child opens the file with `create`, `write`, and `truncate` flags, then uses `dup2()` to replace file descriptors 1 and 2.

```
int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0666);
dup2(fd, STDOUT_FILENO);
dup2(fd, STDERR_FILENO);
ls -la /tmp > output.txt
```

- The command must appear on the left side.
- Exactly one filename must appear on the right side.
- More than one `>` operator is an error.
- Built-in redirection is outside the required scope.

4.7 Parallel commands

Commands separated by `&` are started before the shell waits. This is different from running each command and immediately waiting for it.

```
sleep 2 & echo hello & ls
```

28. Parse one command segment.
29. Fork the external command and store its PID.
30. Move to the next segment without waiting.
31. After every segment has been processed, call `waitpid()` for each stored PID.

4.8 Error behavior

Wish uses one standard error message for shell-detected problems:

```
An error has occurred
```

- Invalid shell startup arguments or an unreadable batch file cause exit status 1.
- Most command errors print the message and allow the shell to continue.
- Examples include invalid built-in arguments, bad redirection syntax, missing commands, `chdir` failure, and `fork/open` errors.

4.9 Compilation and example use

```
gcc -o wish wish.c -Wall -Werror -std=gnu11
./wish
wish> path /bin /usr/bin
wish> pwd
wish> cd /tmp
wish> pwd
wish> ls -la > listing.txt
wish> sleep 1 & echo finished
wish> exit
```

4.10 Suggested tests

- Run a normal command with and without arguments.
- Change the path to empty and verify that external commands stop working.
- Restore the path and run commands again.
- Try exit with an extra argument and cd with zero or two arguments.
- Test redirection with missing filenames, extra filenames, and multiple > symbols.
- Run several sleep commands in parallel and confirm the total time is close to the longest single sleep.
- Run the same commands from a batch file and confirm no prompt appears.

4.11 What this project practices

- Process creation and replacement
- Parent/child synchronization
- Executable path searching
- Built-in shell commands
- File-descriptor redirection
- Basic command parsing and parallelism

5. Final Notes

Together, the three projects cover a useful progression in C. Reverse focuses on memory and files. The Unix utilities add command-line conventions, text processing, and binary formats. Wish brings those basics together with operating-system process control.

5.1 Main lessons

- C programs need explicit error checks; failures are usually reported through return values rather than exceptions.
- `stdin`, `stdout`, and `stderr` are normal file streams and can be passed to many of the same functions as opened files.
- Dynamic input is easier to handle with tools such as `getline()` and `realloc()`.
- Binary formats must be read back using the same sizes and order used when writing them.
- `fork()` creates a new process, while `execv()` replaces the child with another program.
- Good command-line programs depend on exact output, predictable return codes, and careful testing.

5.2 Recommended submission check

32. Compile every source file using the exact flags required by the assignment.
33. Run the normal examples and all required error cases.
34. Check that error messages go to `stderr` and match exactly.
35. Use tools such as `diff` or `cmp` to compare expected and actual output.
36. Run a memory checker such as Valgrind when available.
37. Submit only the requested source files with the correct filenames.

```
valgrind --leak-check=full ./reverse input.txt
valgrind --leak-check=full ./wish batch.txt
```

Final takeaway: The programs are small enough to understand end to end, but they use the same core ideas found in larger Unix software: streams, processes, file descriptors, memory, and strict interfaces.